

```

class Solution {
public:
    vector<string> generateParenthesis(int n) {
        //边界判断, 如果n==0 直接返回空
        if(n == 0) return {" "};
        //弄一个哈希表, 存储上一轮的括号样式
        unordered_set<string> prev;
        //第一次插入, 先插入一对括号
        prev.insert("()");
        //讨论接下来每轮的括号插在哪里, 并且哈希表自动去重
        for(int k = 2; k <= n; ++k)
        {
            //已prev为基准, 讨论现在这一轮的括号插入样式
            unordered_set<string> curr;
            //取出上一轮的括号
            for(const string& s : prev)
            {
                int len = s.size();
                //向每个地方插入
                for(int i = 0; i <= len; ++i)
                {
                    string ns = s;
                    ns.insert(i, "()");
                    //插入后自动去重
                    curr.insert(ns);
                }
            }
            //更新prev
            prev.swap(curr);
        }
        //全部插入完毕, 返回数组
        return vector<string>(prev.begin(), prev.end());
    }
}

```

采用哈希表去重的方法减少分支, 逐层判断